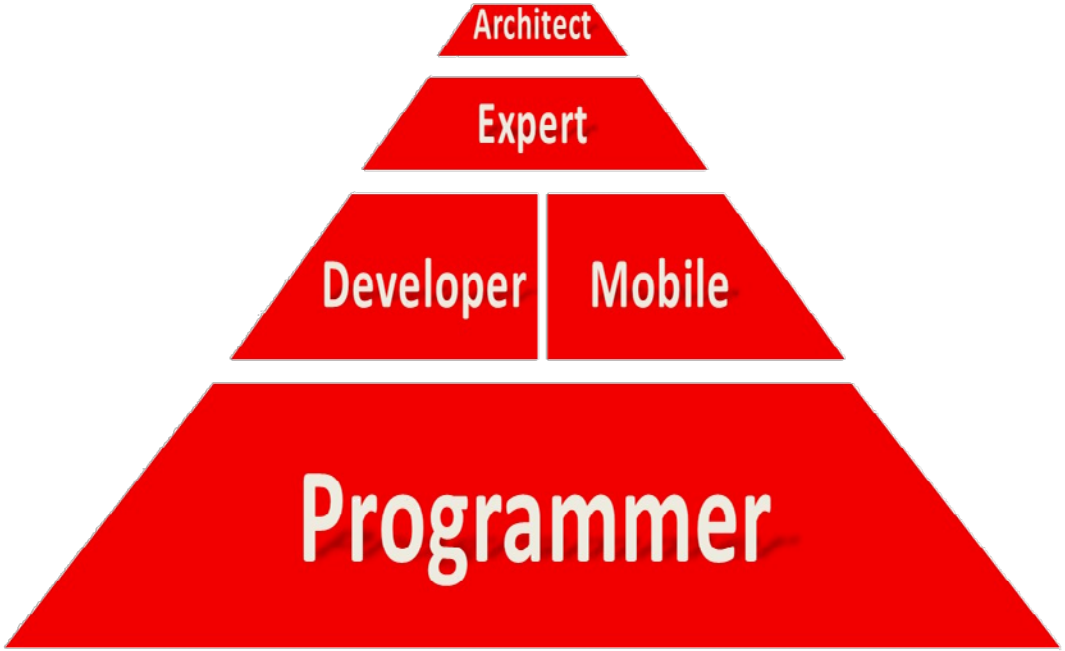
Edwin Maravi
emaravi@cjavaperu.com

CJAVA

Capacitaciones

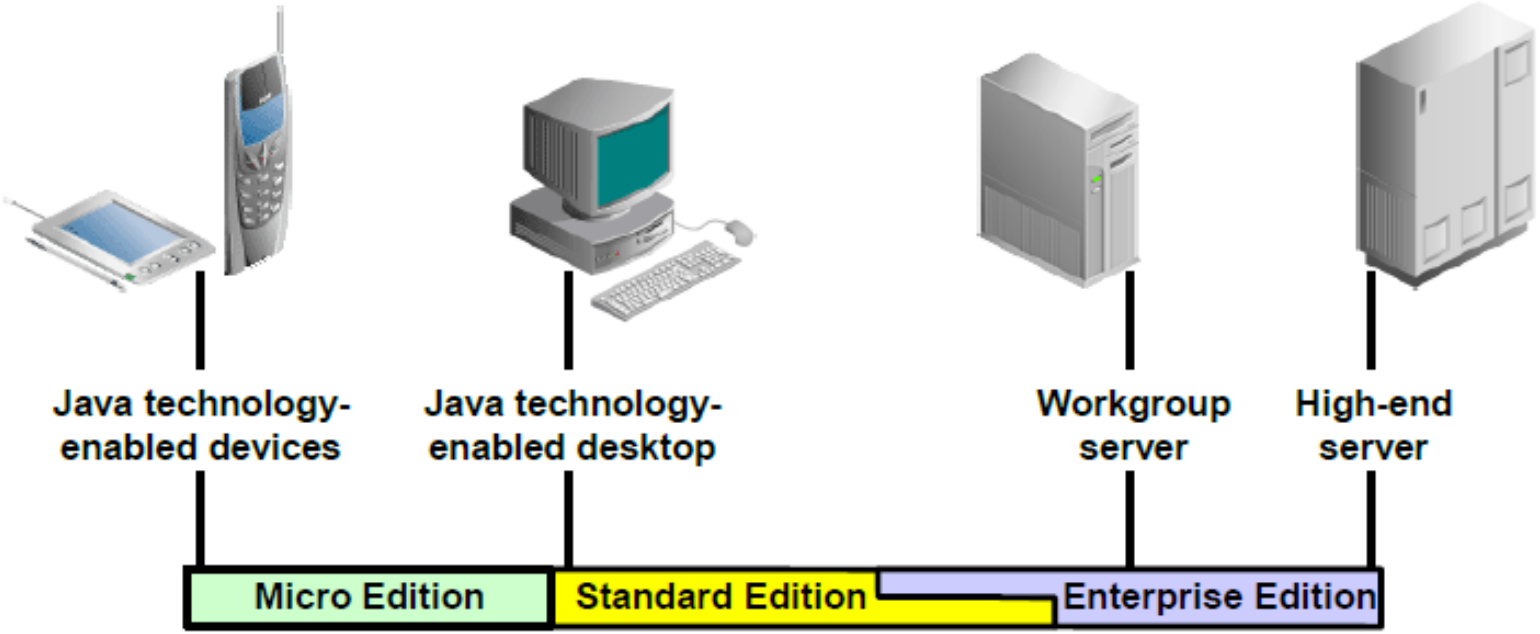


Introducción a la tecnología Web con Java

Edwin Maravi

emaravi@cjavaperu.com

Plataforma Java



Java EE = Java Enterprise Edition = JEE

Necesidades de una aplicación empresarial

Build from the ground up

Developer's Checklist

- [ ] Business services
- [ ] Persistence
- [ ] Transaction management
- [ ] Multithreading
- [ ] Security management
- [ ] Networking
- [ ] Service publishing

Use Application Component Server

Developer's Checklist

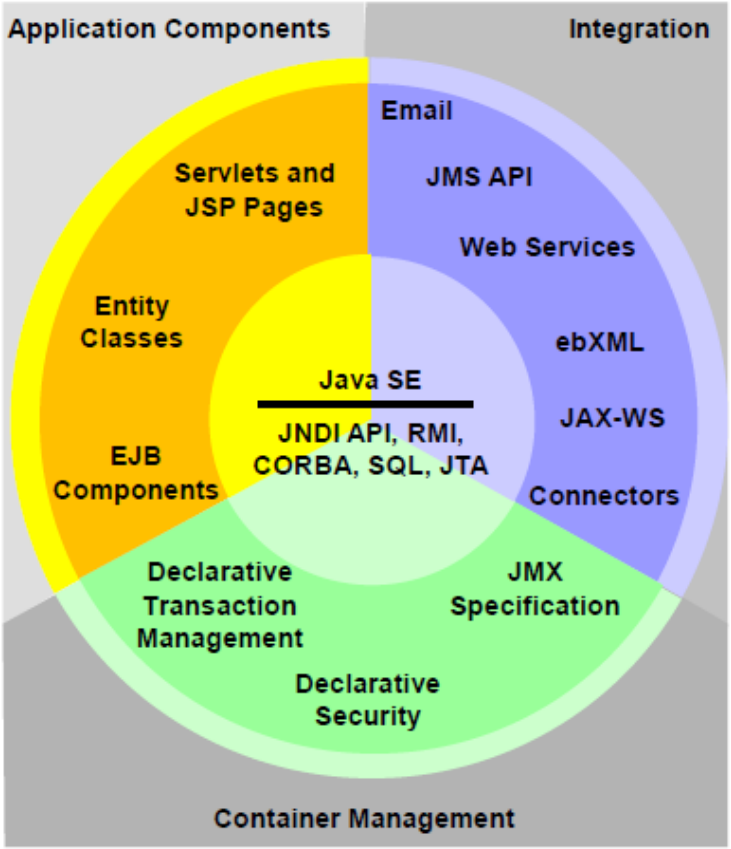
- [ ] Business services

Services Provided by Server

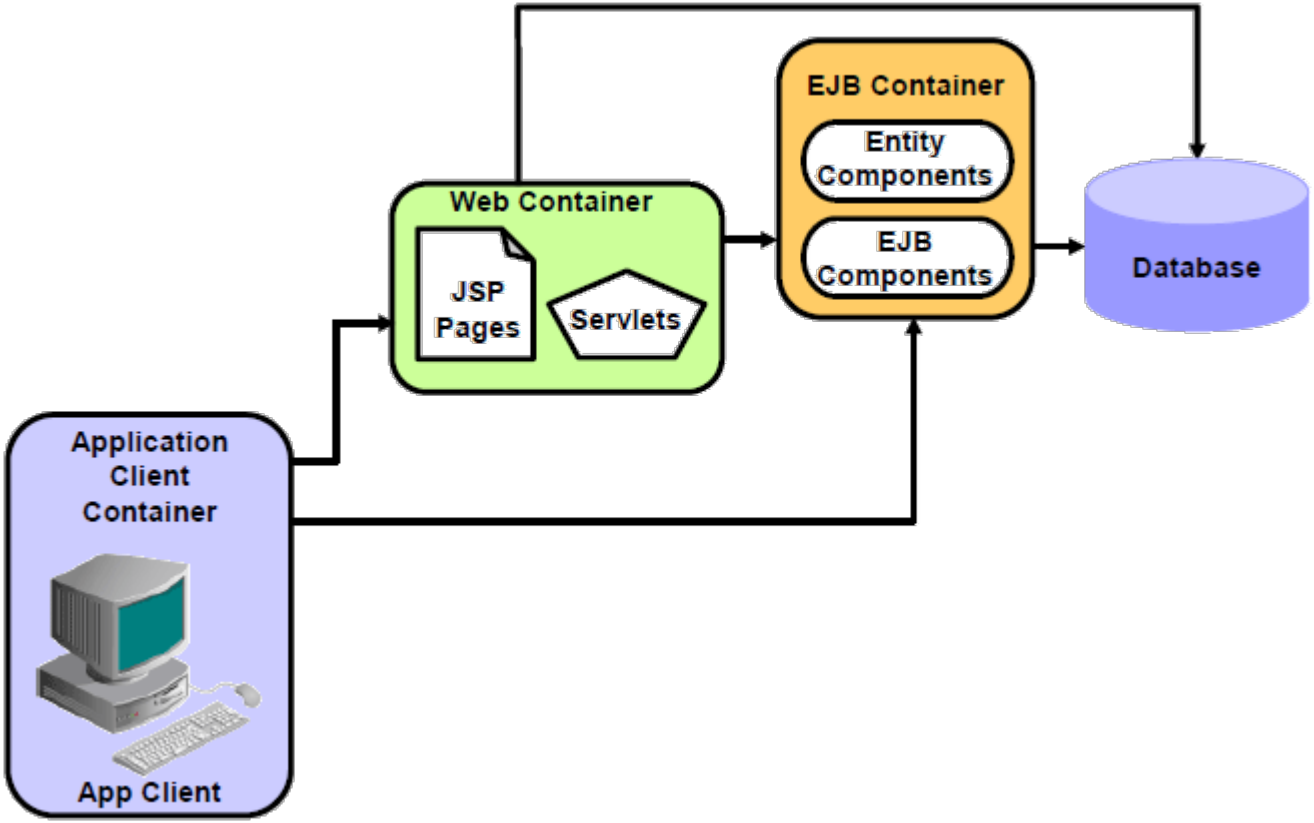
- [x] Persistence and Transaction management
- [x] Multithreading
- [x] Security management
- [x] Networking
- [x] Service publishing

Lo proporciona JEE

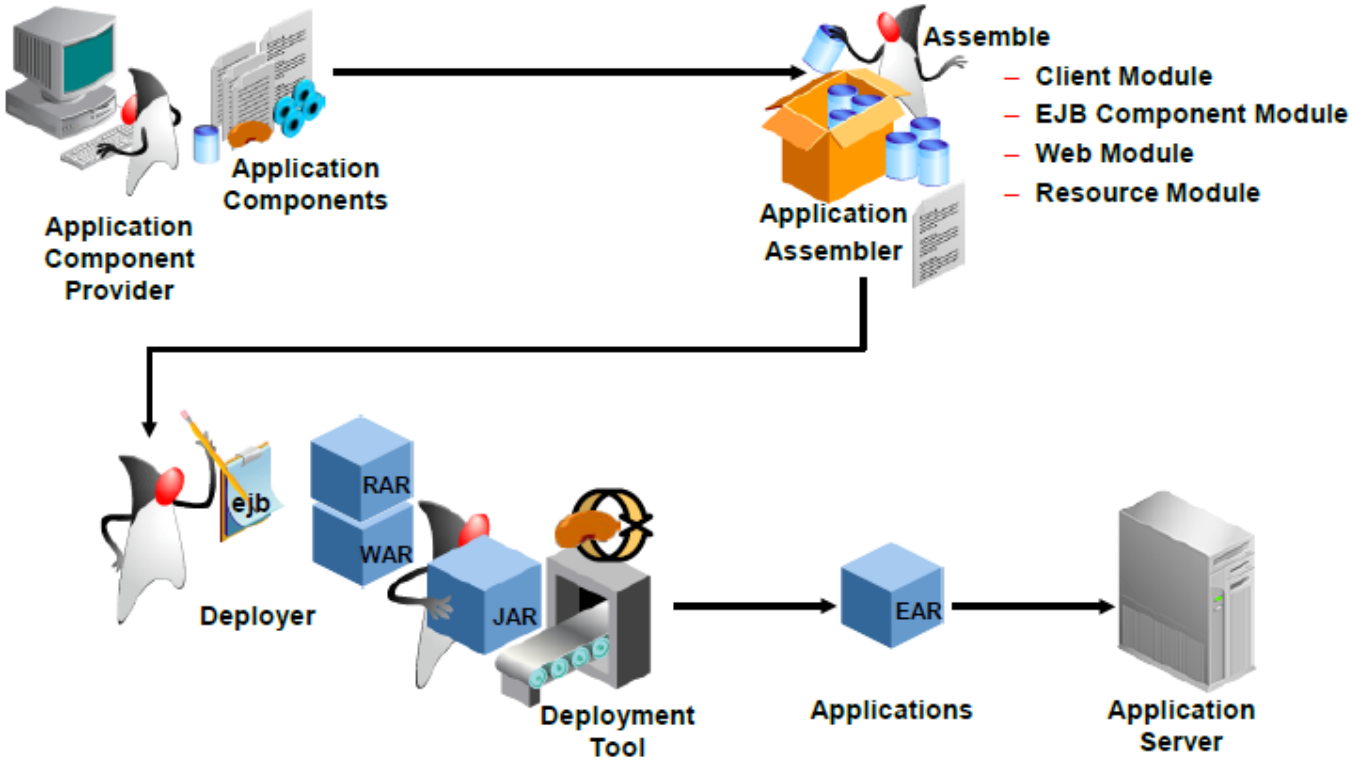
Java Enterprise Edition (JEE)



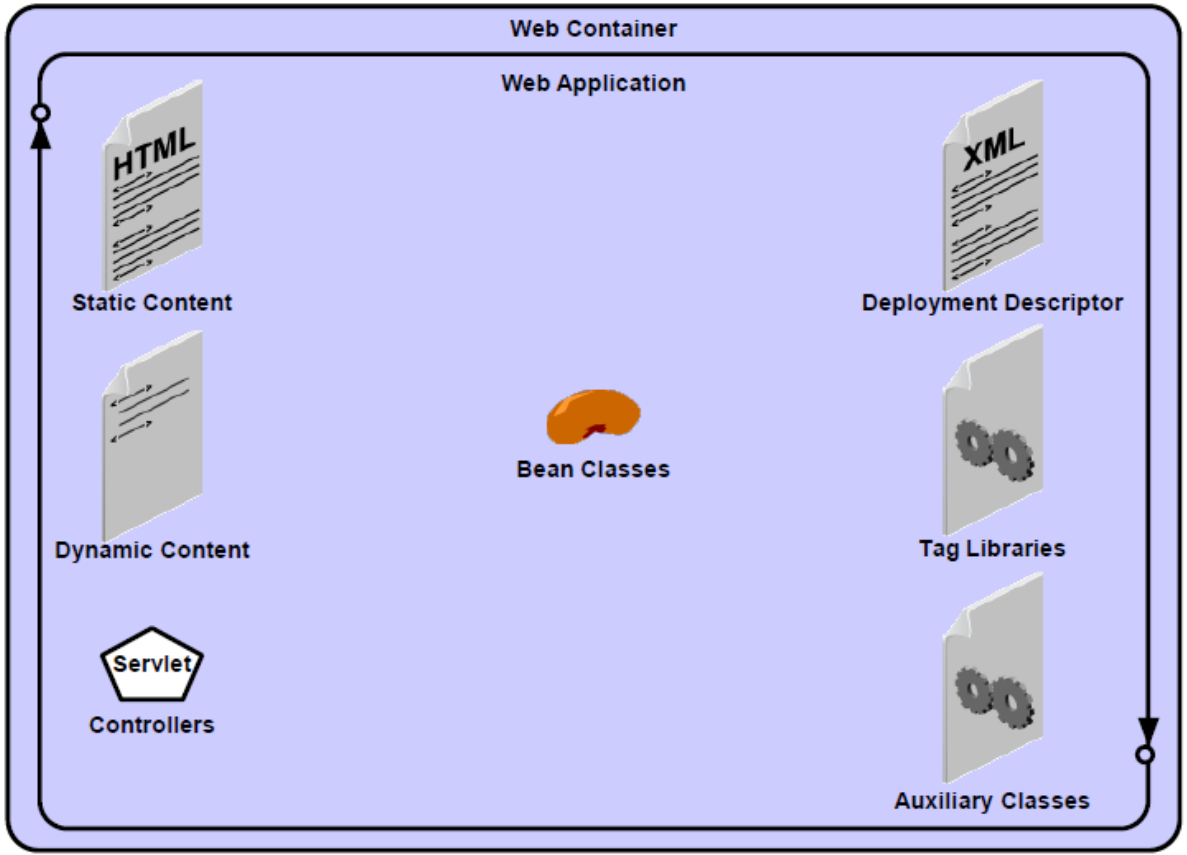
Algunos componentes de JEE



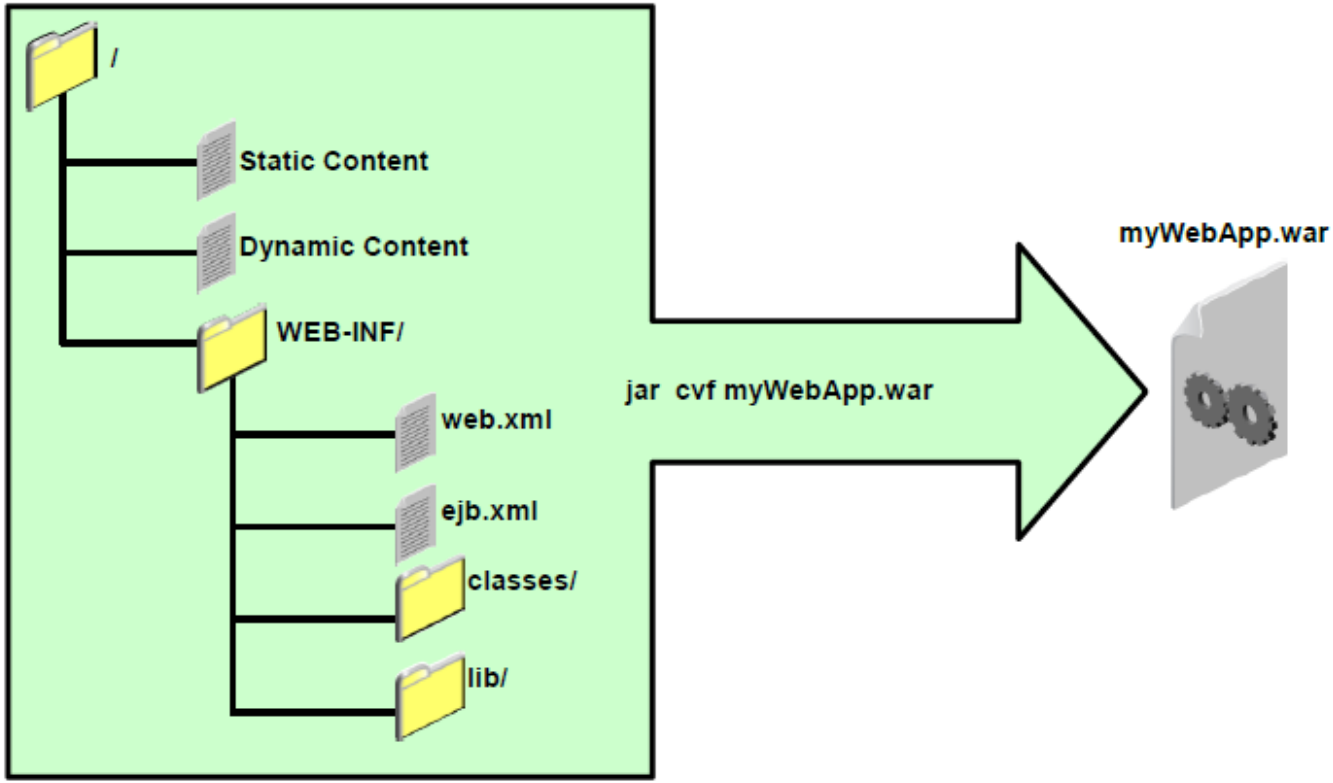
Proceso de desarrollo en JEE



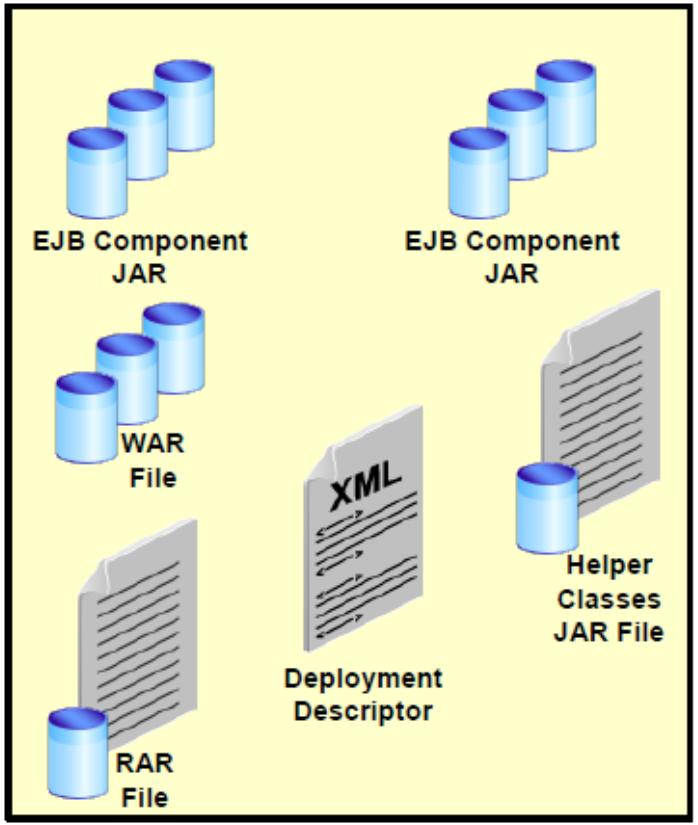
Aplicacion Web (WAR)



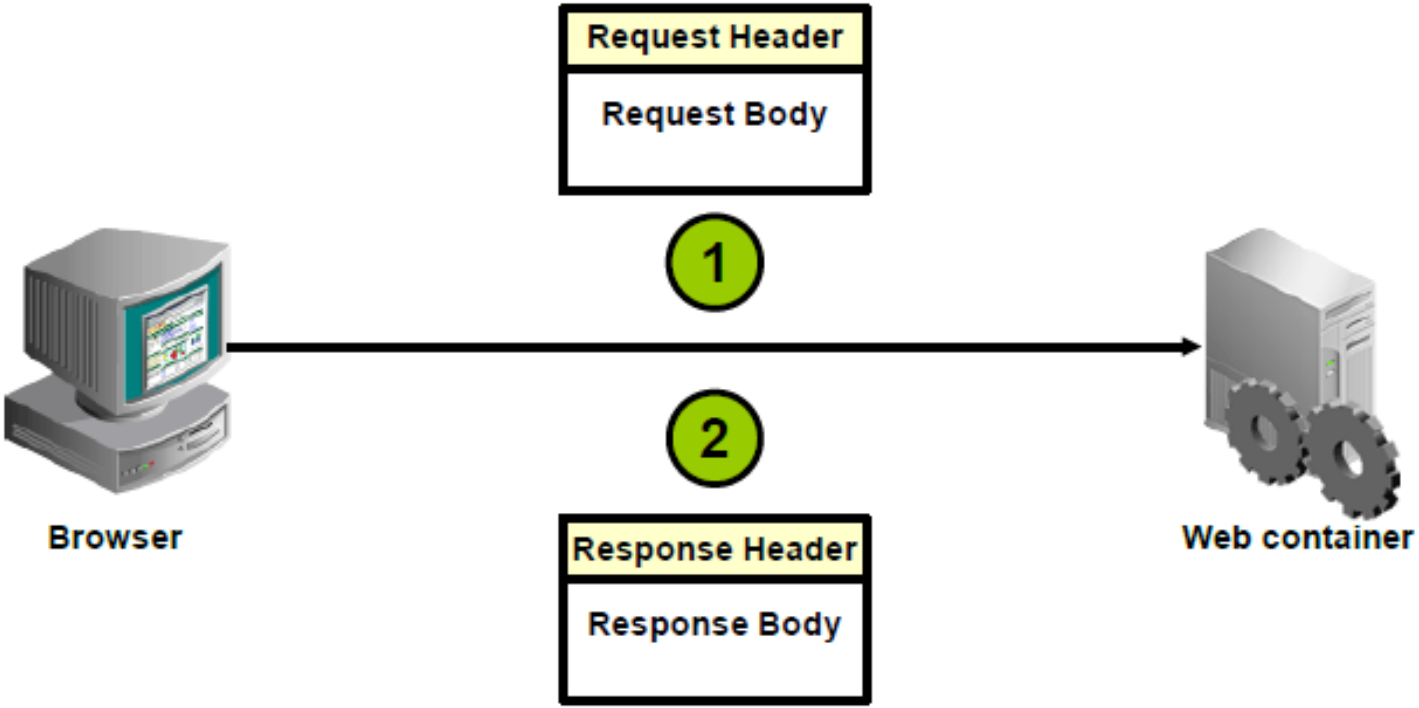
Estructura de una aplicación Web



Aplicación empresarial (EAR)



Protocolo HTTP



Metodos de envío GET y POST

	GET Request	POST Request
Type of Use	Default	Form submission
Method of Sending Form Data	• Sent with the URI • Size limited	• Sent in the request body • Size unlimited
Display of Form Data	Browser is displayed in the URI area.	Browser is not normally displayed with the URI.

Formularios HTML

HTML snippet:

```
<FORM ACTION='form_test' METHOD='POST'>
<INPUT NAME='input1' SIZE='20'/>
<INPUT TYPE='SUBMIT' VALUE='OK'/>
</FORM>
```

Browser form:

Browser request:

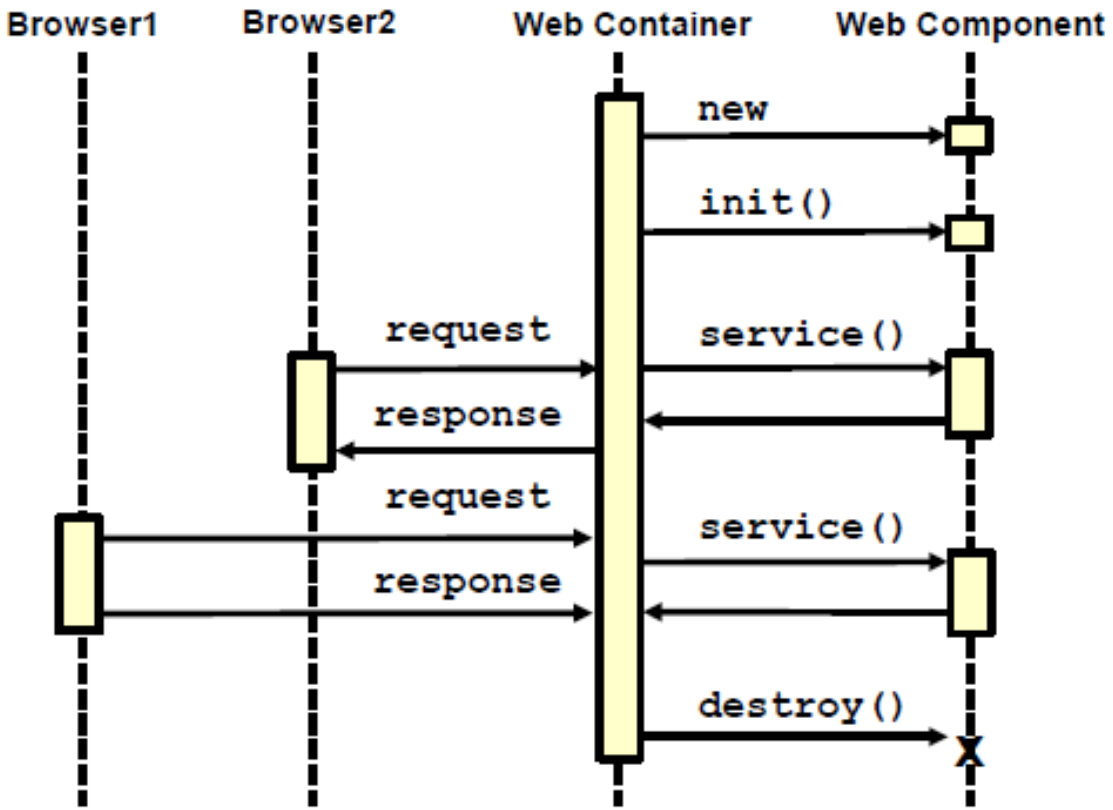
```
POST /bank/form_test HTTP/1.1
... request headers...

input1=this+is+a+test
```

Servlet vs JSP

	Servlets	JSP Components
Description	Web components authored in the Java programming language	Presentation content with embedded programmatic elements
Characteristics	Extend generic base classes in the API, typically the `HttpServlet` interface	• Can be enhanced with custom tags • Are translated into servlets by the web container
Created or Managed By	Developers	Content authors

Ciclo de vida de un componente Web



Ejemplo de Servlet

```
1    package com.example;
2
3    import java.io.*;
4    import java.util.Date;
5    import javax.servlet.annotation.WebServlet;
6    import javax.servlet.http.*;
7
8    @WebServlet("/hello")
9    public class HelloServlet extends HttpServlet {
10
11         @Override
12         protected void doGet(HttpServletRequest request, HttpServletResponse
response) throws IOException {
13              response.setContentType("text/html");
14              PrintWriter out = response.getWriter();
15              out.println("<html><head/><body>");
16              out.println("<h1>Hello, World!</h1>");
17              out.println("The date is:" + new Date());
18              out.println("</body></html>");
19              out.close();
20         }
21   }
```

Ejemplo de JSP

```
1   <%@ taglib uri="http://java.sun.com/jsp/jstl/fmt"
    prefix="fmt"%>
2   <html>
3   <head/>
4   <body>
5   <h1>Hello, World!</h1>
6   <jsp:useBean id="date" class="java.util.Date" />
7   The date is: <fmt:formatDate value="${date}"
    type="both" />
8   </body>
9   </html>
```

Colaboración de JSP y Servlet

	Servlet	JSP Component
Type of Operation	• Process form data • Perform computations • Collect data for rendering	Generate presentation, particularly HTML
Role	Handle requests, perform computations, transfer control to JSP components	Render a response to the initial request

JSPs en tiempo de ejecución

Because JSP components are translated into servlets, JSP components and servlets share runtime behaviors:

- Life cycle and container management
- API and container services
- Client session access

Both can be entered on multiple threads concurrently and must be implemented accordingly.

Web context root y alias mapping

Servlets and JSP components are packaged into a web application.

- Static content, such as HTML or images, is included.
- A web application URI has the following form:

  `http://server:port/context_root/resource`
- The context root maps to a web application.
- An alias maps from a URL to a JSP or servlet.

  `http://www.mybank.com/bank/main`

The `main` resource could be specified:

- In the deployment descriptor using a URL pattern(s)
- In the servlet class using an annotation to specify a URL pattern(s)

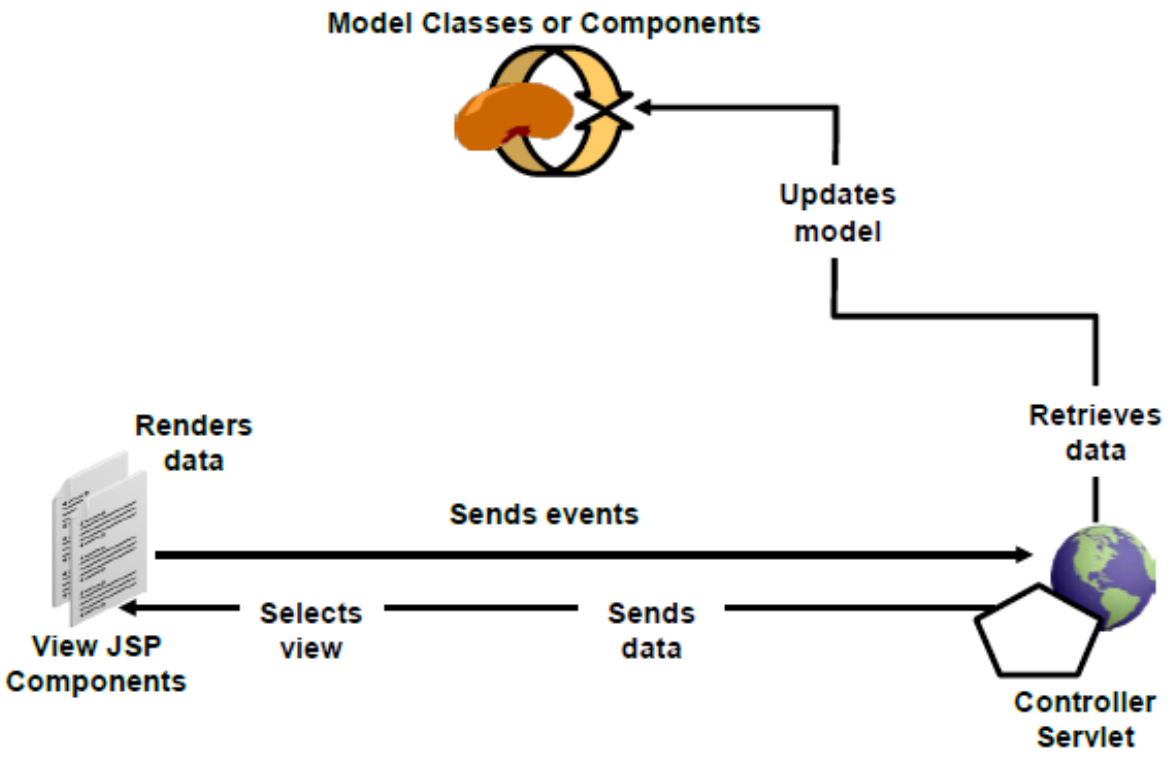
Manejo de sesiones en aplicaciones Web

The HTTP protocol is stateless. Conversational state might be stored on either the browser or the server:

	Browser	Server
Description	Simple and does not consume server resources	Must carry a session ID between the browser and server
Storage Availability	Limited	Less restricted

The Java EE model provides a simple mechanism for storing conversational state on the server.

Patrón de diseño Model View Controller (MVC)



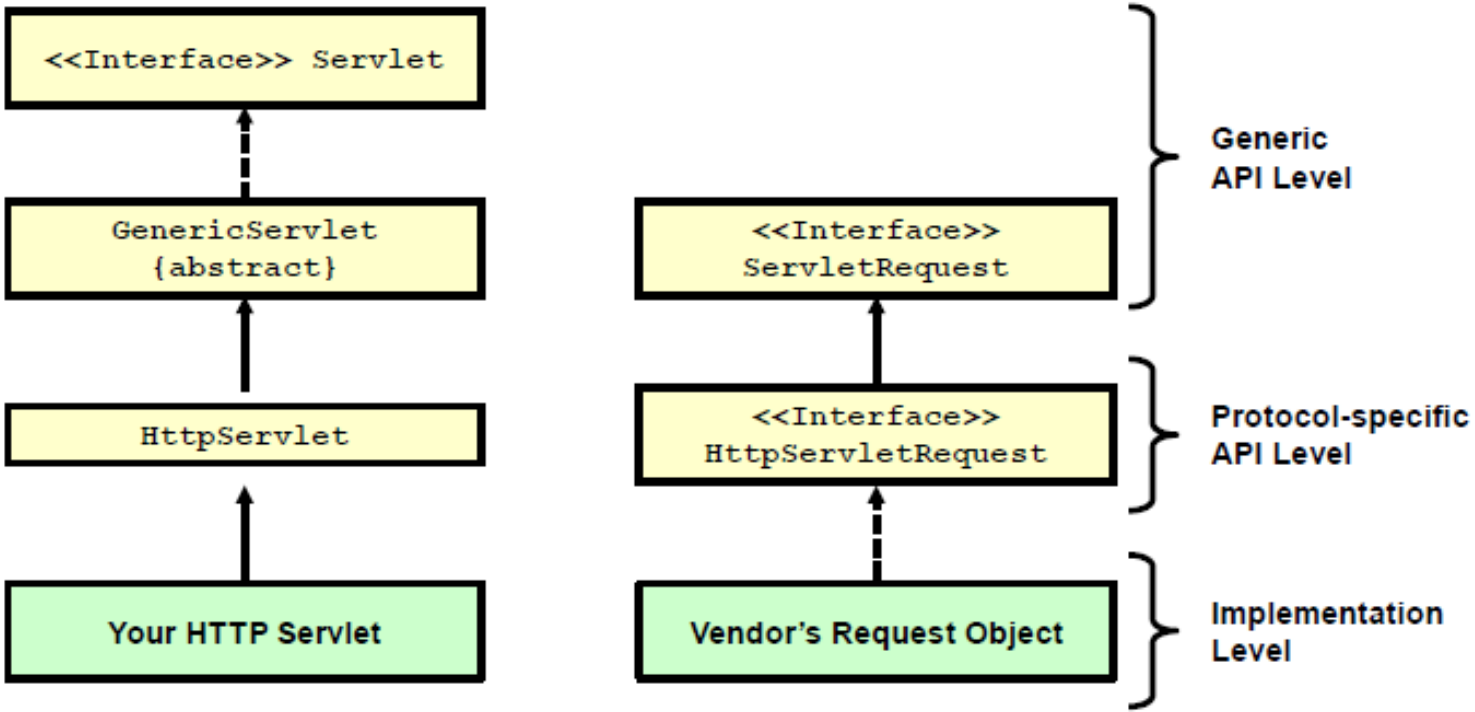
CJAVA
siempre para apoyarte

Aplicaciones Web con Java

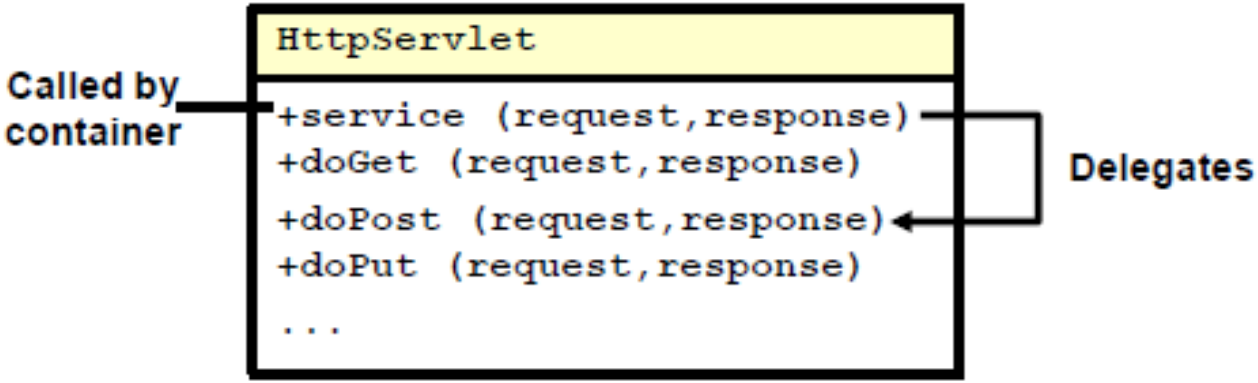
CJAVA
siempre para apoyarte

Java Servlets

El API Servlet



La clase HttpServlet



Ejemplo de Servlet

```
1    package com.example;
2
3    import javax.servlet.annotation.WebServlet;
4    import javax.servlet.http.*;
5
6    @WebServlet("/example")
7    public class ExampleServlet extends HttpServlet {
8
9        @Override
10       public void doGet(HttpServletRequest request, HttpServletResponse response) {
11           processRequest(request, response);
12       }
13
14       @Override
15       public void doPost(HttpServletRequest request, HttpServletResponse response) {
16           processRequest(request, response);
17       }
18
19       public void processRequest(HttpServletRequest request, HttpServletResponse response) {
20           // Process request and generate response
21       }
22   }
```

Configuración de un Servlet

Without configuration, a servlet does not have an accessible URL. Beginning with Servlet spec 3.0 (Java EE 6), annotations are used to map URLs to servlets.

- Single-URL example:

```
@WebServlet("/myservlet")
public class MyHttpServlet extends HttpServlet{
//...
}
```

- Multiple-URL example:

```
@WebServlet(name="SomeName", urlPatterns={"/myservlet",
   "/foo", "/bar"})
public class MyHttpServlet extends HttpServlet{
//...
}
```

Deployment Descriptor de un WAR

To configure a servlet and many other aspects of a web application, with a deployment descriptor place a `web.xml` file in the `WEB-INF` directory.

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="3.0" xmlns="http://java.sun.com/xml/ns/javaee"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://java.sun.com/xml/ns/javaee http://
java.sun.com/xml/ns/javaee/web-app_3_0.xsd">
    <servlet>
        <servlet-name>SomeName</servlet-name>
        <servlet-class>package.MyHttpServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>SomeName</servlet-name>
        <url-pattern>/myservlet</url-pattern>
    </servlet-mapping>
</web-app>
```

Ciclo de vida de un Servlet

As outlined in the previous module, servlets are multithreaded. A single servlet instance is created for each `<servlet>` configured in the `web.xml` file. A servlet is instantiated sometime before its `service` method is called.

A servlet can have two styles of initialization methods that are called before the `service` method and two types of methods called before discarding the servlet.

```
public void init() {....}
@PostConstruct public void myInitMethod() {...}

public void destroy() {...}
@PreDestroy public void myDestroyMethod() {...}
```

La clase HttpServletRequest

Generic Methods	Purpose
`getParameter`	Gets form data elements
`getAttribute` and `setAttribute`	Gets and sets attributes, which are used for passing data between components
`getRequestDispatcher`	Gets a request dispatcher to transfer control to another component
HTTP-Specific Methods	**Purpose**
`getUserPrincipal`	The identity of the user, if authenticated
`getCookies`	Gets the identity of the user, if authenticated
`getSession`	Gets client session

La clase HttpServletResponse

Generic Methods	Purpose
`getOutputStream`, `getWriter`	Gets a stream or writer to send data to the browser
`setContentType`	Indicates the MIME type of response body
HTTP-Specific Methods	**Purpose**
`encodeURL`	Adds a session ID to a URL
`addCookie`	Sends a cookie to the browser
`sendError`	Sends an HTTP error code

Recibiendo datos de un formulario

```
1   public void processRequest (HttpServletRequest request,
    HttpServletResponse response)
2         throws IOException {
3     response.setContentType ("text/plain");
4     PrintWriter out = response.getWriter();
5     String name = request.getParameter ("name");
6     if (name == null || name.length() == 0)
7         name = "anonymous";
8     out.println ("Hello, " + name);
9     out.close();
10      }
```

Pasando el flujo a otro componente Web

There are two `getRequestDispatcher("URI")` methods that return a `RequestDispatcher` implementation.

The `ServletRequest` method that accepts relative paths or paths beginning with a "/"

```
RequestDispatcher requestDispatcher =
request.getRequestDispatcher("relativeURI");
```

The `ServletContext` method that must begin with a "/"

```
RequestDispatcher requestDispatcher
=   getServletContext().getRequestDispatcher
    ("/ServletName");
```

Pasando objetos a otro componente Web

The request object can carry data between components:

- In the calling component:

```
CustomerData customerData = // get customer data
request.setAttribute ("customerData",
customerData);
requestDispatcher.forward (request, response);
```

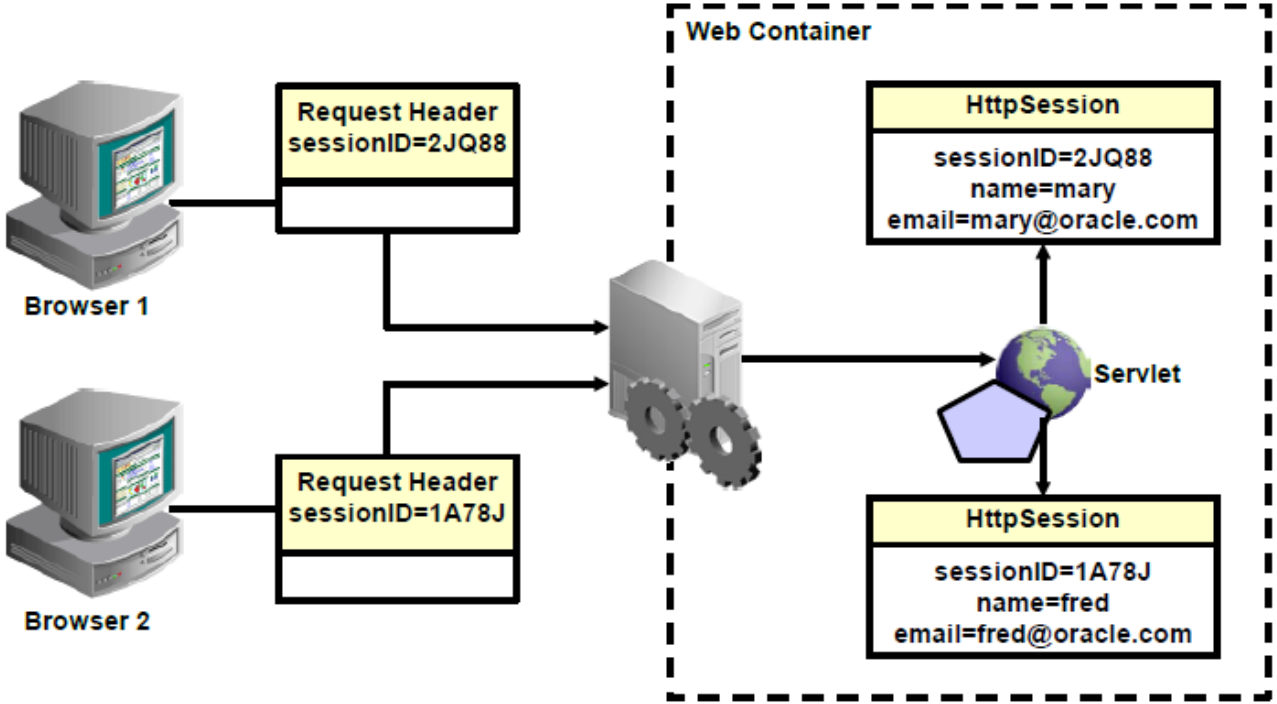
- If the target component is a servlet:

```
CustomerData customerData = (CustomerData)
request.getAttribute("customerData");
```

- If the target component is a JSP component:

```
<jsp:useBean id="customerData"
class="Bank.CustomerData" scope="request"/>
```

Manejo de sesiones en aplicaciones Web



La clase HttpSession

The `request.getSession` method always returns a session. The `HttpSession.isNew` method returns true in either of the following situations:

- The session is a new session with a new browser.
- The current browser session timed out before this request.

To close a session early, call its `invalidate` method:

```
if ("logout".equals(request.getParameter("action")) {
  session.invalidate();
}
```

Revisar sesiones nuevas

```
1   // Get a session object for the current client, creating
2   // a new session if necessary
3   HttpSession session = request.getSession();
4
5   // If this is a new session, initialize it
6   if (session.isNew()) {
7     // Initialize the session attributes
8     // to their start-of-session values
9     session.setAttribute ("account", new Account());
10      // ... other initialization
11    }
12
13    // Get this client's 'account' object
14    Account account = (Account) session.getAttribute("account");
```

CJAVA

siempre para apoyarte

Gracias